Объектно-ориентированное программирование

4 семестр
Лекция 2.

Содержание лекции:

- История создания Git
- Работа в Git
- Конфликт слияния веток
- GitHub

Преподаватель курса: Клюкин Даниил Анатольевич, ст. преподаватель каф. ПМиИТ

Все материалы по ИТиП1-3, ООП1, ООП2 размещены на GitHub:

https://github.com/DaniilKlyukin/ITiP-1

https://github.com/DaniilKlyukin/ITiP-2

https://github.com/DaniilKlyukin/ITiP-3

https://github.com/DaniilKlyukin/OOP-4

https://github.com/DaniilKlyukin/OOP-6

Git

Git – это бесплатная распределенная система контроля версий с открытым исходным кодом, предназначенная для быстрой и эффективной работы со всеми проектами, от небольших до очень крупных.

Применение:

- Командная разработка отдельных модулей;
- Возможность отката к любой из предыдущих версий программы. При этом важно знать, что Git хранит только изменения файлов, а не сами версии файлов, что существенно экономит ресурсы ПК.

История создания Git

Ядро Linux - это достаточно большой проект с открытым исходным кодом. Большую часть времени разработки ядра Linux (1991–2002 гг.) изменения передавались между разработчиками в виде патчей и архивов. В 2002 году проект ядра Linux начал использовать проприетарную децентрализованную систему контроля версий BitKeeper.

В 2005 году отношения между сообществом разработчиков ядра Linux и коммерческой компанией, которая разрабатывала BitKeeper, прекратились, и бесплатное использование утилиты стало невозможным. Это сподвигло сообщество разработчиков ядра Linux (а в частности Линуса Торвальдса — создателя Linux) разработать свою собственную утилиту, учитывая уроки, полученные при работе с BitKeeper.

Некоторыми целями, которые преследовала новая система, были:

- Скорость
- Простая архитектура
- Хорошая поддержка нелинейной разработки (тысячи параллельных веток)
- Полная децентрализация
- Возможность эффективного управления большими проектами, такими как ядро Linux (скорость работы и разумное использование дискового пространства)

Официальный сайт программы: https://git-scm.com/

Литература (полезные ссылки):

1. https://ru.hexlet.io/courses/intro_to_git
2. https://githowto.com/ru/setup
3. https://learngitbranching.js.org/?locale=ru_RU (тренажер по Git)
4. https://www.youtube.com/watch?v=zZBiln_2FhM (видео-урок)

Термины:

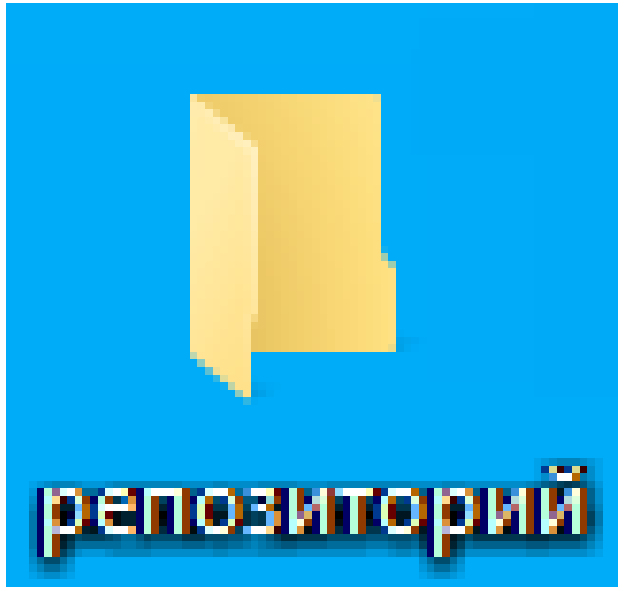
Репозиторий – любая папка, в которой, например, находится наш проект.

Ветка (Branch) – представляет собой какую-то версию нашего проекта (репозитория).

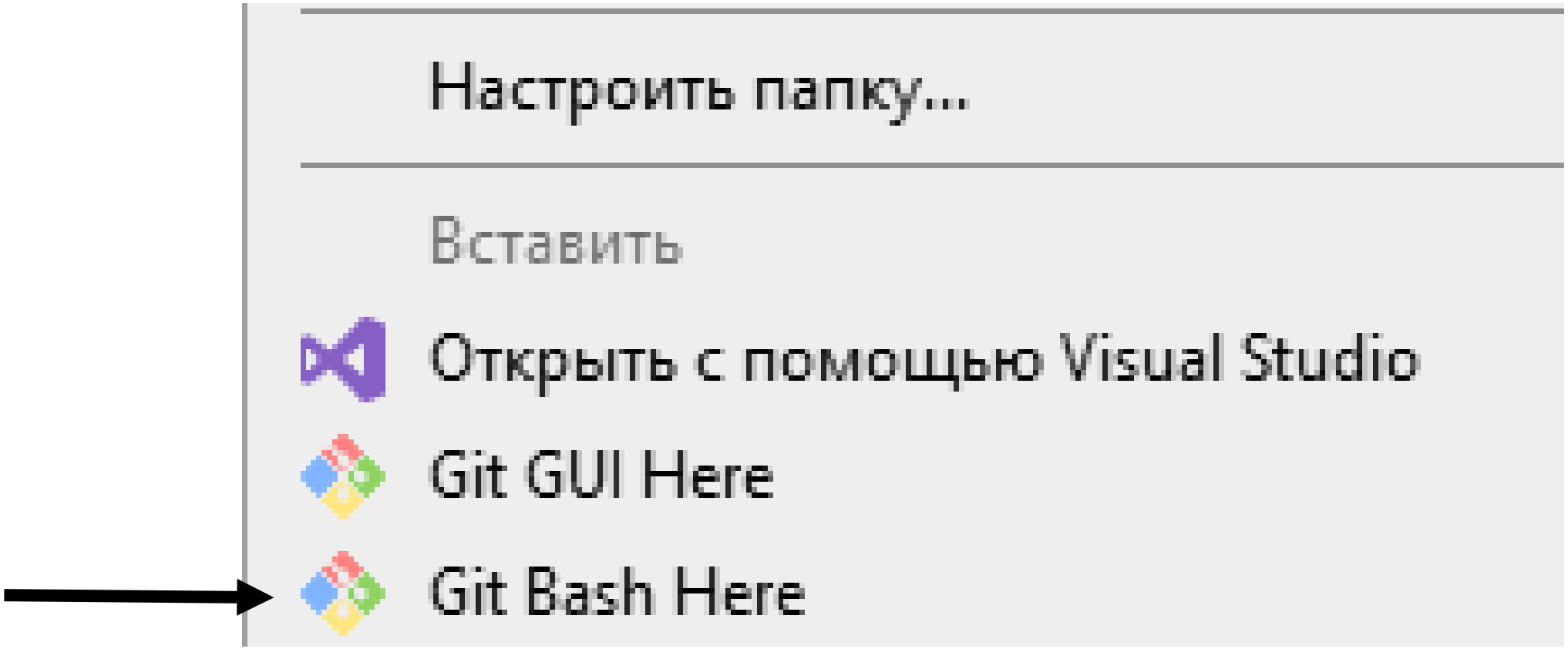
Шаги по работе с Git:

Устанавливаем Git (все опции оставляем по умолчанию).

Создаем репозиторий (папку).



Заходим в папку, нажимаем ПКМ и выбираем **Git Bash Here**.



MINGW64:/c/Users/anato/OneDrive/Рабочий стол/репозиторий

```
anato@DESKTOP-MOMT9UL MINGW64 ~/OneDrive
/Рабочий стол/репозиторий
```

Для инициализации репозитория пишем команду **git init**

```
$ git init
Initialized empty Git repository in C:/Users/
anato/OneDrive/Рабочий стол/репозиторий/.git/

anato@DESKTOP-MOMT9UL MINGW64 ~/OneDrive/Рабо
чий стол/репозиторий (master)
$
```

В репозиторий можно поместить абсолютно любой файл или папку.

Создадим в репозитории текстовый файл (file 1.txt):

Закроем файл и с помощью команды **git status** посмотрим на состояние репозитория.

Введем команду **git status**

MINGW64:/c/Users/anato/OneDrive/Рабочий стол/репозиторий

```
anato@DESKTOP-MOMT9UL MINGW64 ~/OneDrive/Рабочий сто
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will b
e committed)
        file 1.txt

nothing added to commit but untracked files present
(use "git add" to track)
```

On branch master говорит о том на какой ветке мы находимся (master).

No commits yet означает, что мы ещё не фиксировали изменения в репозитории.

Untracked files содержит список файлов, которые были изменены или добавлены, но эти действия ещё не добавлены в список для фиксации.

В данном случае это файл **file 1.txt**

Nothing added to commit говорит о том, что мы ничего не добавили для фиксации изменений в репозитории.

Добавим изменения командой **git add «название файла или папки»**, чтобы добавить все изменения необходимо ввести **«git add .»**

```
anato@DESKTOP-MOMT9UL MINGW64 ~/OneDrive/Рабочий сто
л/репозиторий (master)
$ git add .
```

Если ввести **git status**, то можно увидеть какие файлы ожидают фиксации (сейчас они ещё не сохранены).

```
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   file 1.txt
```

Зафиксируем изменения (сохраним) с помощью команды **git commit –m «Любой текст, описание комита»**.

```
anato@DESKTOP-MOMT9UL MINGW64 ~/OneDrive/Рабочий сто
л/репозиторий (master)
$ git commit -m v1
[master (root-commit) 766c3ef] v1
 1 file changed, 1 insertion(+)
 create mode 100644 file 1.txt
```

Фиксация изменений выполнена успешно, 1 файл изменен.

Можно делать множество изменений, удалять, добавлять, изменять файлы.

Игнорирование файлов, файл gitignore

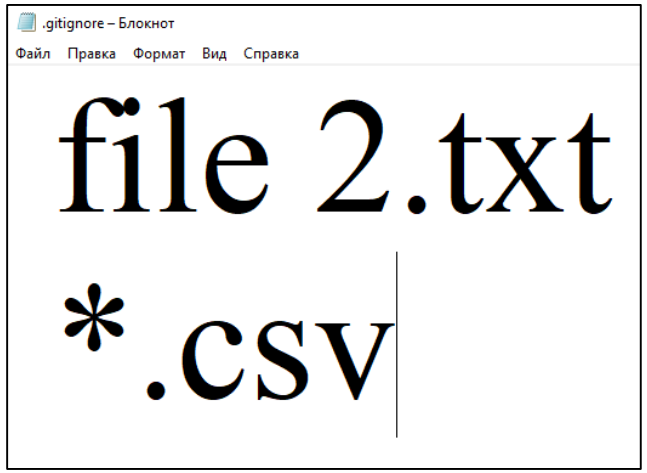
Чтобы не отслеживать изменения некоторых файлов, необходимо создать специальный файл с расширением .gitignore

В данном файле перечисляются шаблоны (или конкретные названия) файлов, которые не будут отслеживаться git.

Создадим такой файл со следующим содержимым:

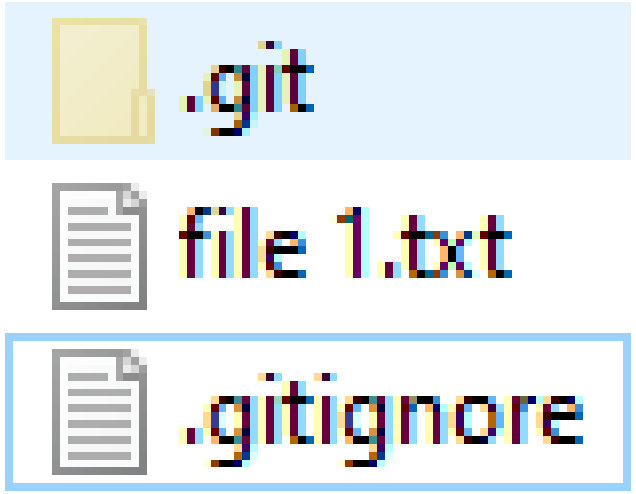
в репозиторий не попадет файл с именем file 2.txt, а также файлы с расширением .csv

ОБЯЗАТЕЛЬНО нужно зафиксировать gitignore файл отдельным комитом ДО добавления файлов, которые нужно игнорировать.



Зафиксируем изменения с помощью команд add и commit.

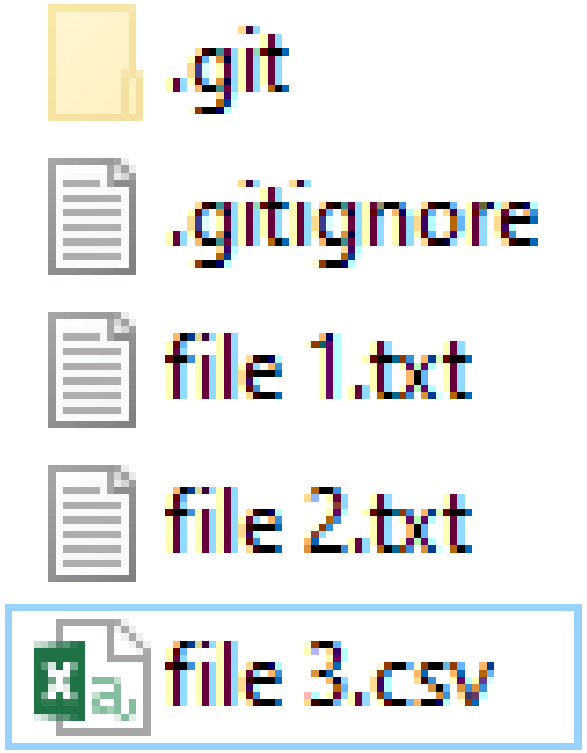
```
anato@DESKTOP-MOMT9UL MINGW64 ~/OneDrive/Рабочий сто
л/репозиторий (master)
$ git commit -m "add gitignore"
[master befd06e] add gitignore
 1 file changed, 2 insertions(+)
 create mode 100644 .gitignore
```



Далее эти команды можно совмещать с помощью одной команды:

git commit -am «add gitignore»

Теперь попробуем добавить файлы, которые должны игнорироваться




```
anato@DESKTOP-MOMT9UL MINGW64 ~/OneDr
л/репозиторий (master)
$ git status
On branch master
nothing to commit, working tree clean
```

Видим, что изменения репозитория не найдены. file 2.txt и file 3.csv не обнаруживаются git-ом.

Ветки

Ветка представляет собой независимое направление разработки.

Допустим мы хотим добавить новый функционал в наш проект (программу), но не уверены, понравится ли он заказчику, поэтому в случае чего хотим отменить изменения. Для этого существуют ветки, обычно основная версия программа хранится в ветке main или master, а разработчики создают отдельные ветки, в которых работают над программой, добавляют функционал, изменяют программу, что-то тестируют.

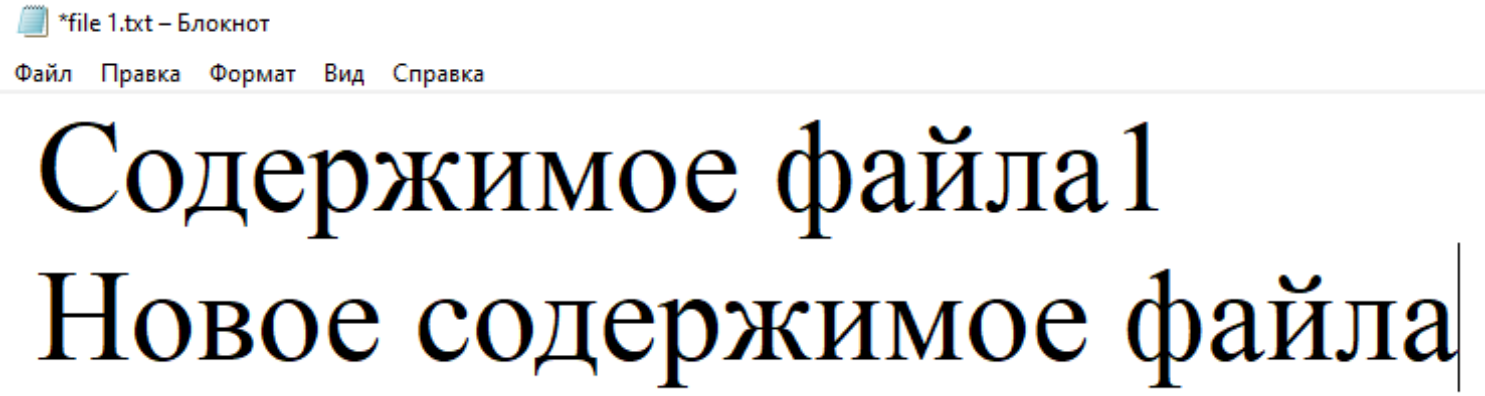
Чтобы создать ветку напишем команду **git branch test** и перейдем на данную ветку командой **git checkout test**

```
anato@DESKTOP-MOMT9UL MINGW64 ~/OneDrive/Рабочий стол
/репозиторий (master)
$ git checkout test
Switched to branch 'test'

anato@DESKTOP-MOMT9UL MINGW64 ~/OneDrive/Рабочий стол
/репозиторий (test)
$ |
```

Заметим, что теперь в скобках отображается **test** – текущая ветка.

Изменим наш файл file 1.txt



Фиксируем изменения **git commit -am «edit file2»**

```
anato@DESKTOP-MOMT9UL MINGW64 ~/OneDrive/Рабочий стол
/репозиторий (test)
$ git commit -am "edit file2"
[test 2dea8d9] edit file2
 1 file changed, 2 insertions(+), 1 deletion(-)

anato@DESKTOP-MOMT9UL MINGW64 ~/OneDrive/Рабочий стол
/репозиторий (test)
$
```

Чтобы наши изменения перешли в ветку master необходимо перейти в master и выполнить слияние с веткой test:

git checkout master

git merge test

```
anato@DESKTOP-MOMT9UL MINGW64 ~/OneDrive/Рабочий стол
/репозиторий (master)
$ git merge test
Updating befd06e..2dea8d9
Fast-forward
 file 1.txt | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
```

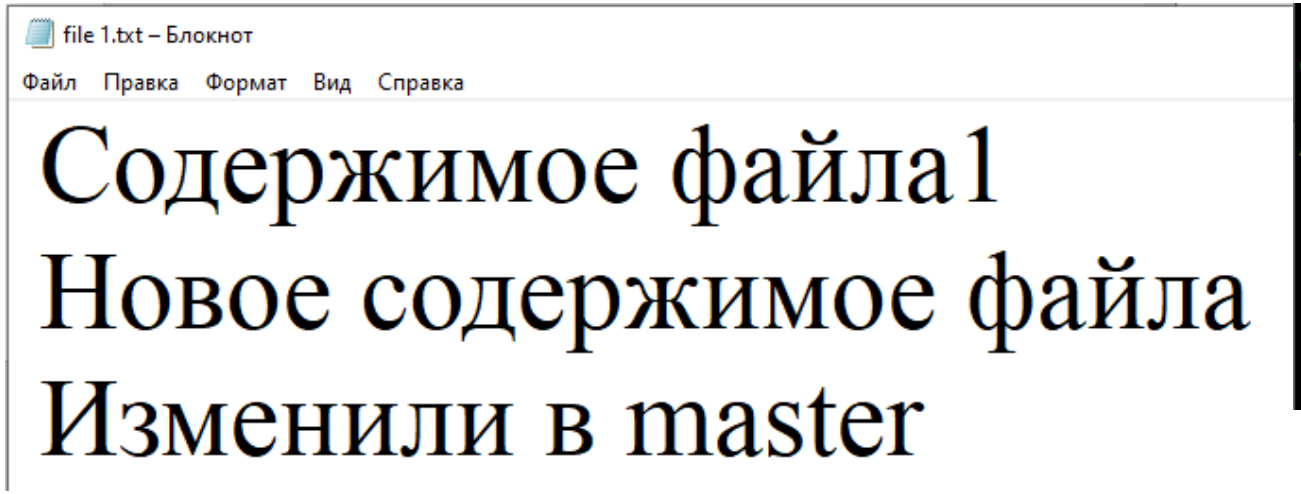
Теперь ветка master содержит изменения из test. Если бы мы решили, что изменения из test нам не нужны (неудачная версия проекта), то мы бы не делали слияние.

Конфликт слияния веток

Конфликт слияния веток возникает только тогда, когда был изменен разным способом один и тот же файл в одной и той же ветке, либо когда производится слияние веток с разными изменениями одного и того же файла.

Смоделируем конфликт слияния:

в ветке master изменим file 1.txt и зафиксируем изменения




```
anato@DESKTOP-MOMT9UL MINGW
/репозиторий (master)
$ git commit -am "v2"
[master b607959] v2
 1 file changed, 2 insertic
```

в ветке test изменим file 1.txt другим образом и зафиксируем изменения

file 1.txt – Блокнот

Файл Правка Формат Вид Справка

Содержимое файла1
Новое содержимое файла
Добавили в test

```
anato@DESKTOP-MOMT9UL MINGW64 ~/OneDrive/Рабочий стол
/репозиторий (test)
$ git commit -am "v2 test"
[test 7f921c5] v2 test
 1 file changed, 2 insertions(+), 1 deletion(-)
```

Теперь перенесем изменения из test в master. Для этого переходим в ветку master и делаем слияние: **git checkout master** **git merge test**

```
anato@DESKTOP-MOMT9UL MINGW64 ~/OneDrive/Рабочий стол
/репозиторий (master)
$ git merge test
Auto-merging file 1.txt
CONFLICT (content): Merge conflict in file 1.txt
Automatic merge failed; fix conflicts and then commit
 the result.

anato@DESKTOP-MOMT9UL MINGW64 ~/OneDrive/Рабочий стол
/репозиторий (master|MERGING)
```

Сейчас мы находимся в процессе слияния, нужно вручную выполнить слияние веток. Из текста видно, что конфликтует файл file 1.txt

Заходим в конфликтующий файл.

Видим, что в файле отображаются изменения из обеих веток,

нам исправить файл так как мы считаем нужным. Конфликт считается решенным, если в файле нет спец. строк

<<<<<<< HEAD

=======

>>>>>>> test

file 1.txt – Блокнот

Файл Правка Формат Вид Справка

Содержимое файла1
Новое содержимое файла
<<<<<<< HEAD
Изменили в master
=======
Добавили в test
>>>>>>> test

Допустим, что мы хотим оставить и те и другие изменения, просто сотрем эти спец. строки.

После редактирования файла получили следующее:

file 1.txt – Блокнот

Файл Правка Формат Вид Справка

Содержимое файла1
Новое содержимое файла
Изменили в master
Добавили в test

Далее сохраняем файл и завершаем слияние командой add+commit:

git commit -am "merged"

```
anato@DESKTOP-MOMT9UL MINGW64 ~/OneDrive/Рабочий стол
/репозиторий (master|MERGING)
$ git commit -am "merged"
[master 5f301e0] merged
```

Программирование и git

Рассмотрим этапы работы с git при написании программ.

Ещё раз отметим, что репозиторий это обычная папка, в которой может храниться все что угодно.

Однако при написании программ, например, на C# в папке с проектом создается огромное количество вспомогательных файлов, которые пересоздаются при компиляции и которые не нужно сохранять в репозитории.

Поэтому **важнейшим этапом** здесь является добавление **gitignore** файла с правильным содержимым. Добавить его нужно **ДО** создания программы в репозитории. Данный файл для C# можно скачать по ссылке https://github.com/github/gitignore/blob/main/VisualStudio.gitignore

Очистим репозиторий, добавим gitignore файл и зафиксируем изменения.

```
anato@DESKTOP-MOMT9UL MINGW64 ~/OneDrive/Рабочий стол
/репозиторий (master)
$ git commit -am "VS"
[master 44b1fd0] VS
 2 files changed, 6 deletions(-)
 delete mode 100644 .gitignore
 delete mode 100644 file 1.txt
```

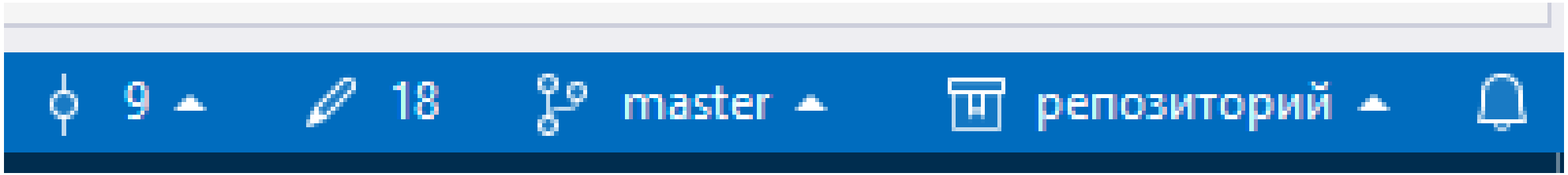
Далее создадим проект, в качестве местоположения проекта выберем репозиторий.

```
1   Console.WriteLine("Hello, World!");
```

Зеленая полоска слева означает, что этой строки не было и она добавлена.

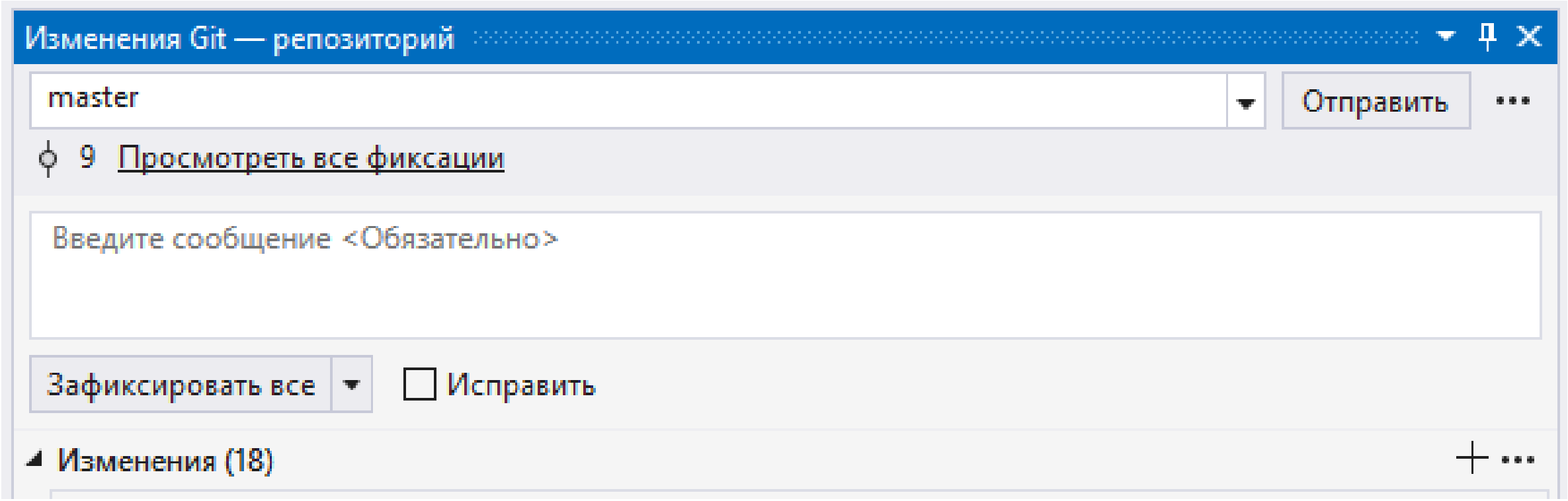
В Visual Studio есть интерфейс для работы с git.

В правой нижней части экрана отображается: номер фиксации, количество изменений, текущая ветка, имя репозитория:



Также есть вкладка Изменения Git (находится в панели «Вид»)

Откроем вкладку Изменения Git



Здесь с помощью графического интерфейса можно сменить ветку и зафиксировать изменения.

Далее рассмотрим другие способы работы с git.

git GUI

Для **git** существует графический интерфейс, который можно вызвать выбрав пункт (при нажатии ПКМ, см. пункты ранее):

Также существует отдельное приложение **GitHub**, которое позволяет сохранять проекты в удаленном репозитории (облачно), отображать статистику вашей работы, делиться проектами с другими людьми, оно позволяет работать в команде удаленно (каждый загружает свои изменения в удаленный репозиторий и загружает в свой локальный репозиторий чужие изменения).

GitHub

GitHub — крупнейший веб-сервис для хостинга IT-проектов и их совместной разработки. Веб-сервис основан на системе контроля версий Git и разработан на Ruby on Rails и Erlang компанией GitHub Inc.

Для начала работы необходимо зарегистрироваться на сайте https://github.com/

и скачать **GitHub Desktop** – приложение, которое позволяет взаимодействовать с GitHub с помощью графического пользовательского интерфейса. https://desktop.github.com/

Создадим репозиторий с помощью **GitHub Desktop** .

Запускаем **GitHub Desktop**, нажимаем File -> New repository, выбираем имя репозитория и местоположение.

Create a new repository

Name

myrepo

Local path

C:\Users\anato\OneDrive\Рабочий стол\repo

Choose...

☐ Initialize this repository with a README

Git ignore

None

License

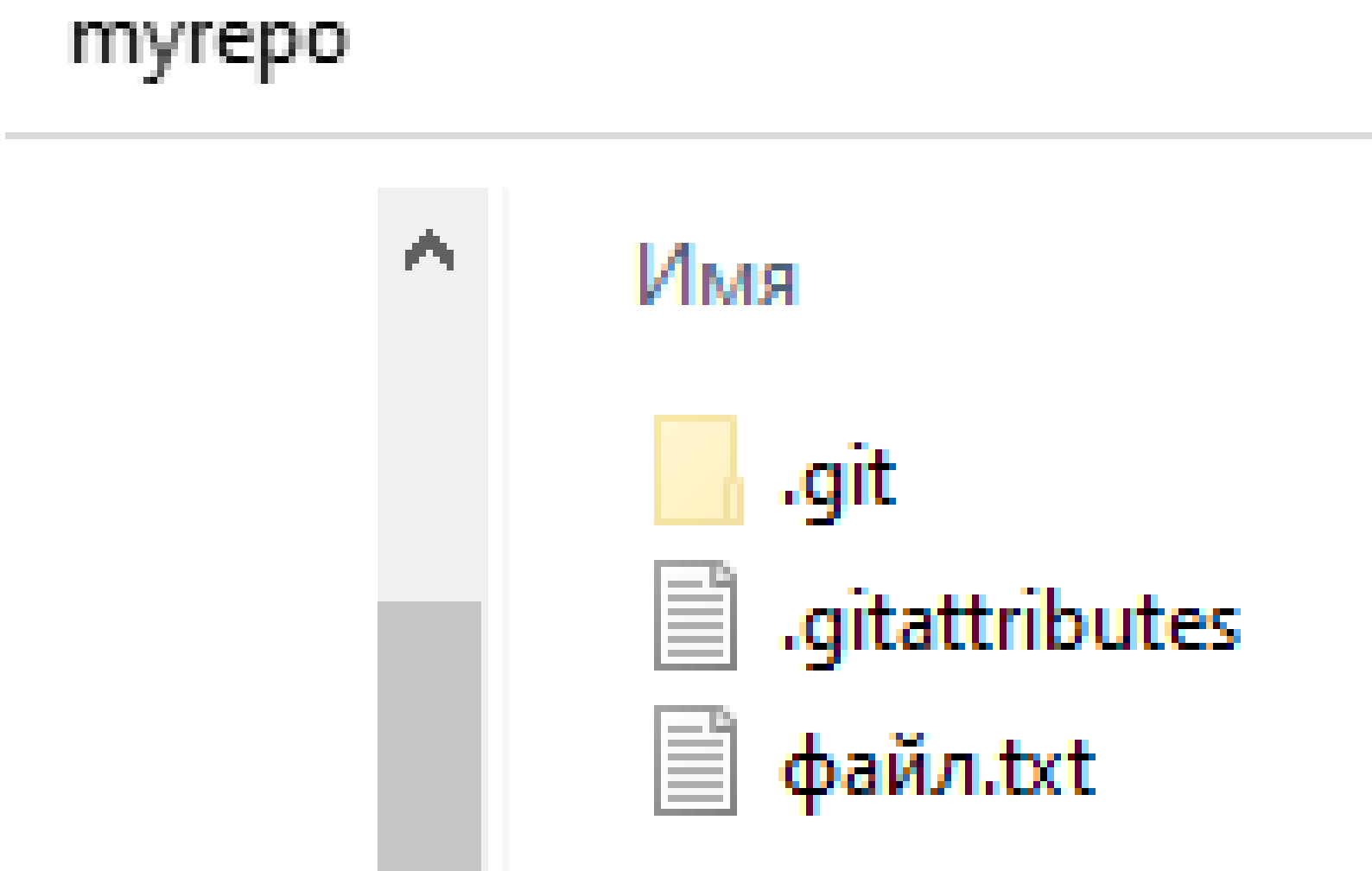
None

Create repository

Cancel

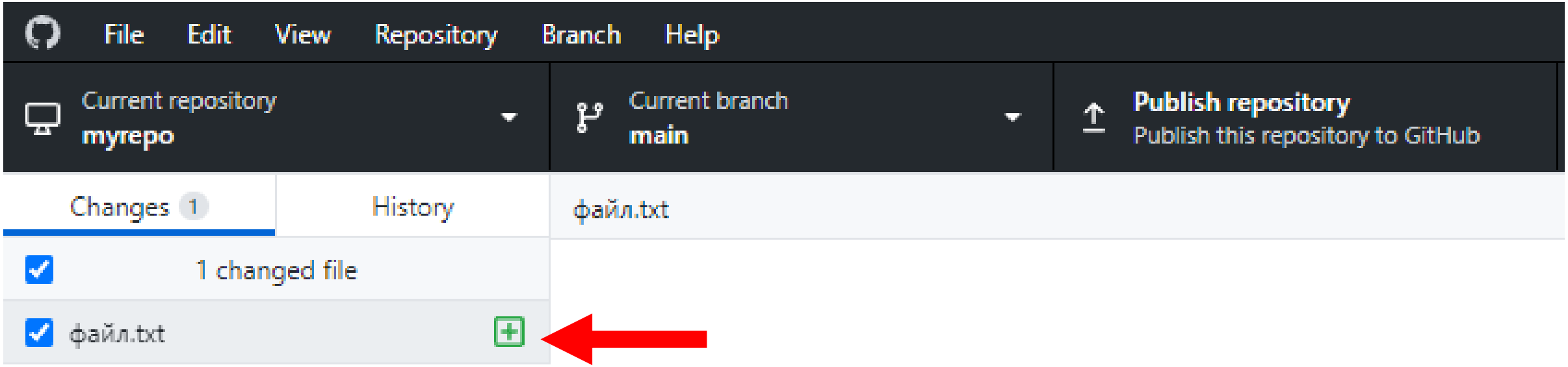
Если это проект на языке C#, то не забываем скачать gitignore для C# и положить в репозиторий.

Для примера добавим текстовый файл в наш репозиторий.

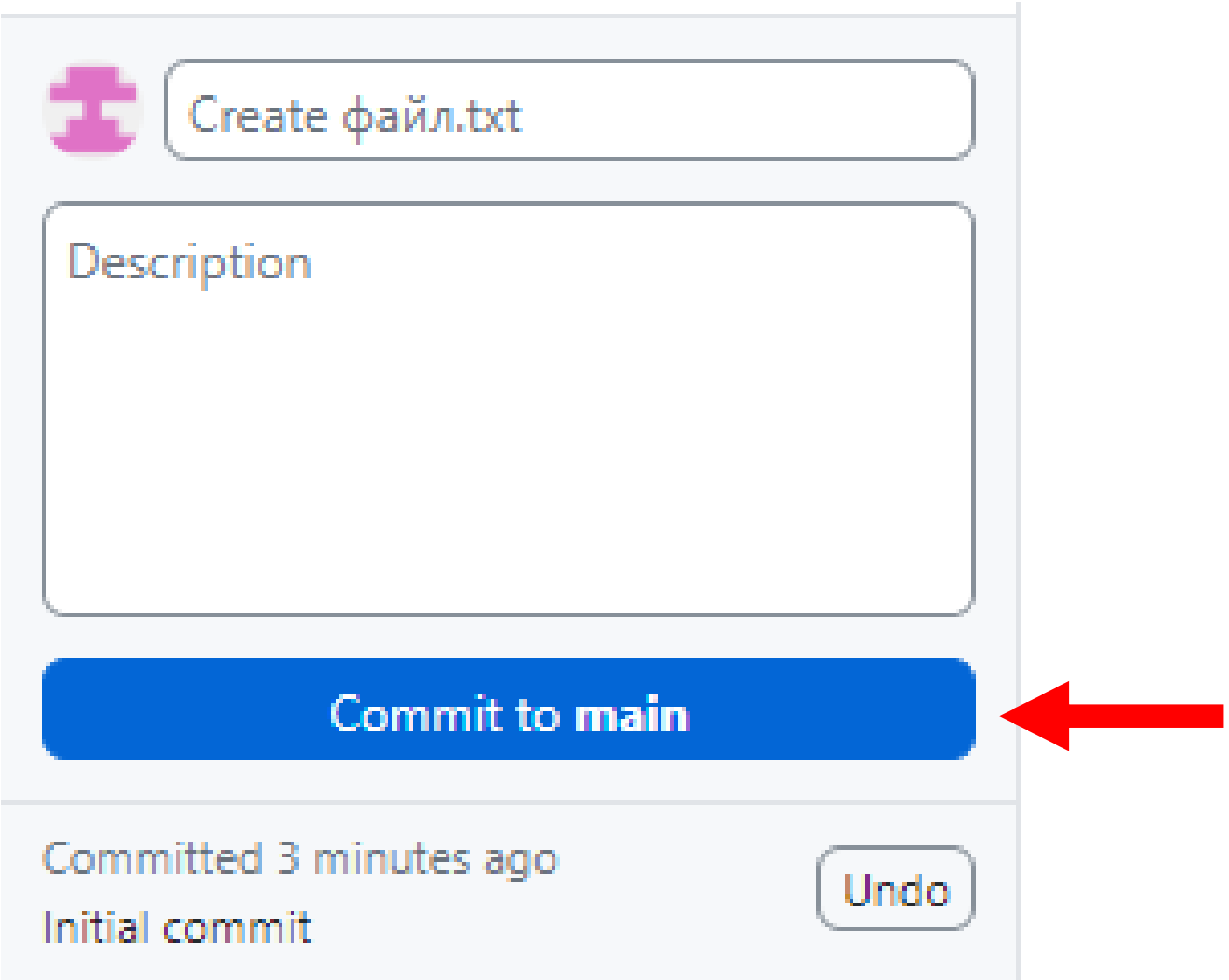


Зайдем в **GitHub Desktop**

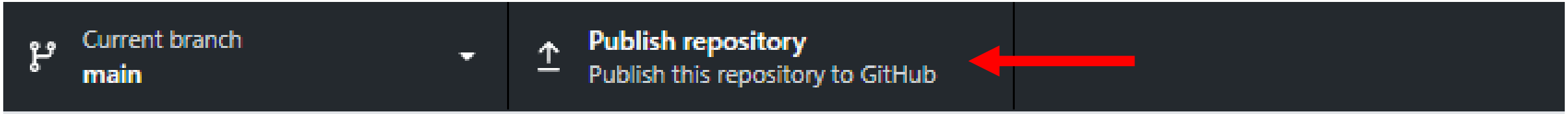
В программе зеленым плюсом отмечено, что было добавлено, видим, что добавился «файл.txt».

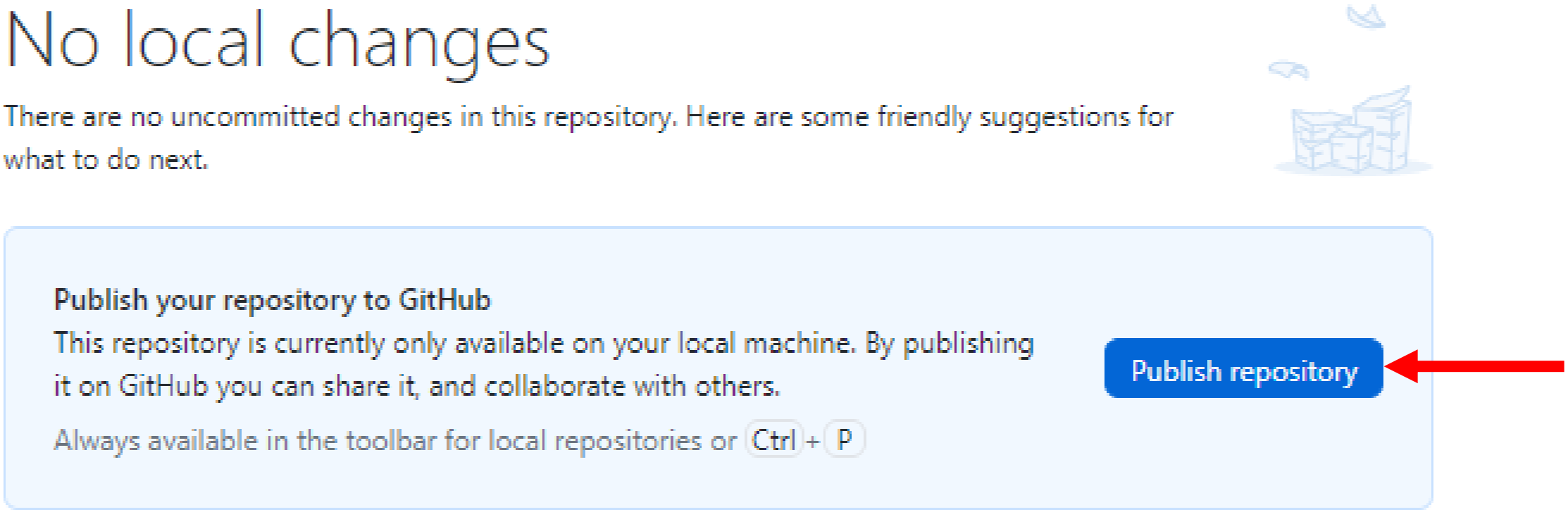


Зафиксируем изменения с помощью комита, введем название комита и нажмем **Commit to main**.

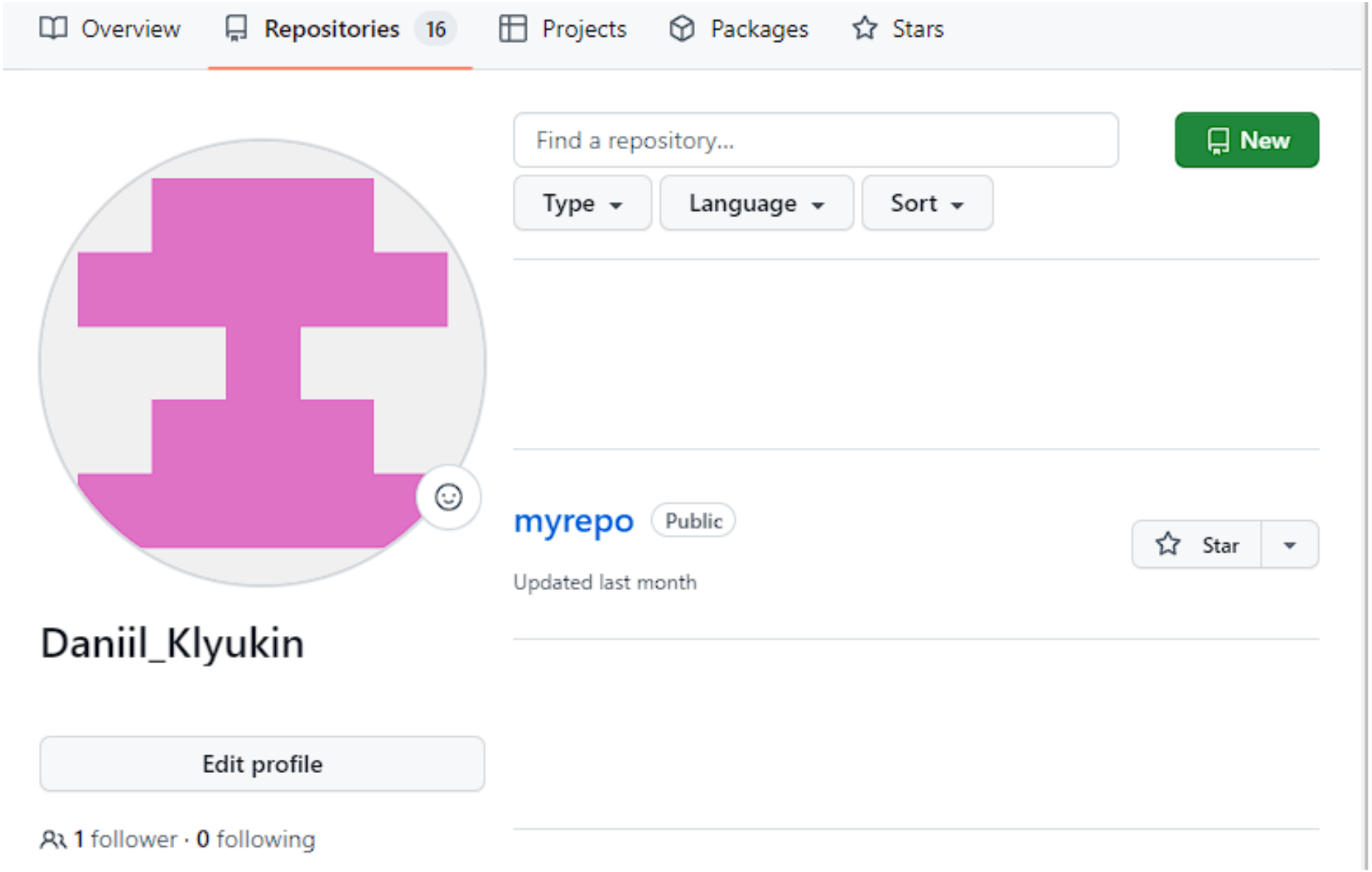


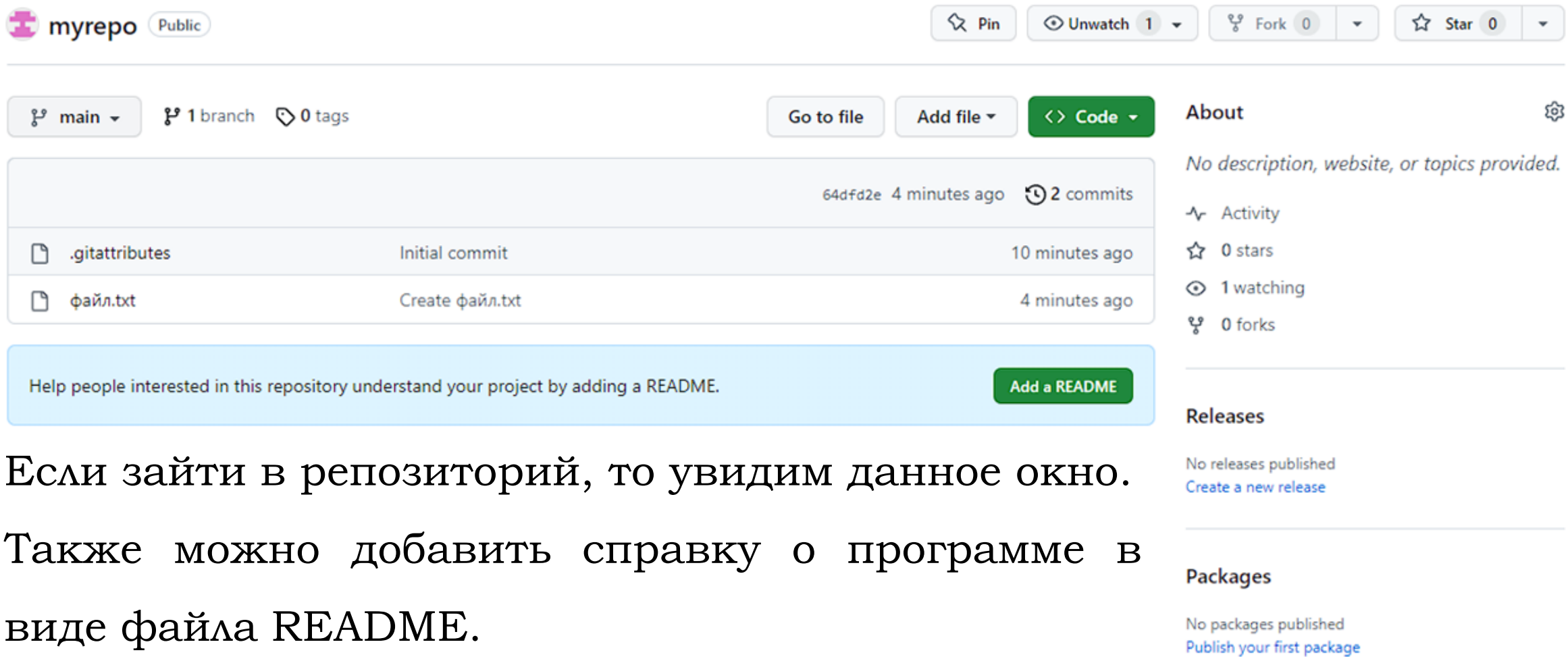
Пока что наш репозиторий локальный, чтобы к нашему репозиторию был доступ из интернета необходимо опубликовать его. Нажмем publish repository





После опубликования наш репозиторий появится у нашего аккаунта GitHub на сайте.

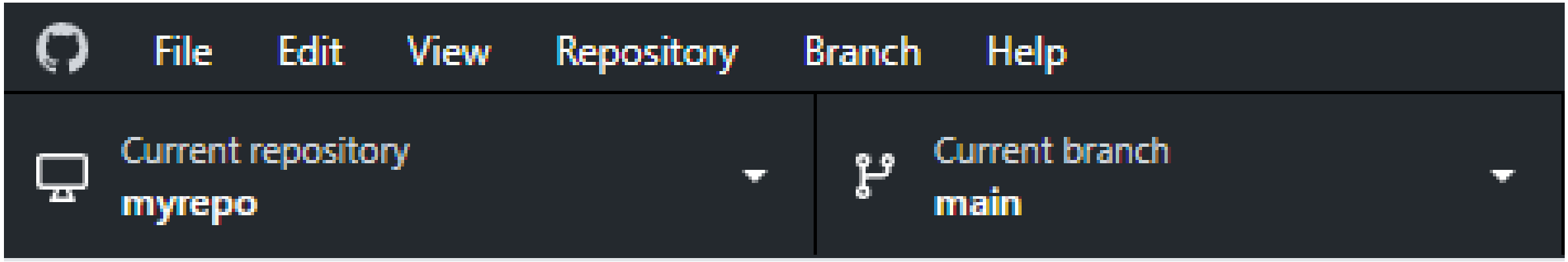


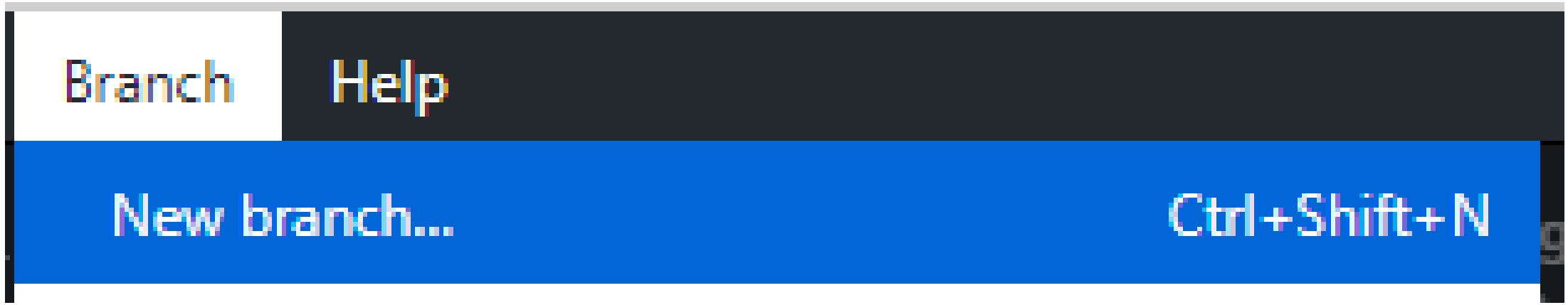


Если зайти в репозиторий, то увидим данное окно. Также можно добавить справку о программе в виде файла README.

Далее алгоритм работы может быть следующий:

Для создания новой ветки необходимо зайти **Branch** -> **New branch**, после чего у вас добавится новая ветка





Create a branch

Name

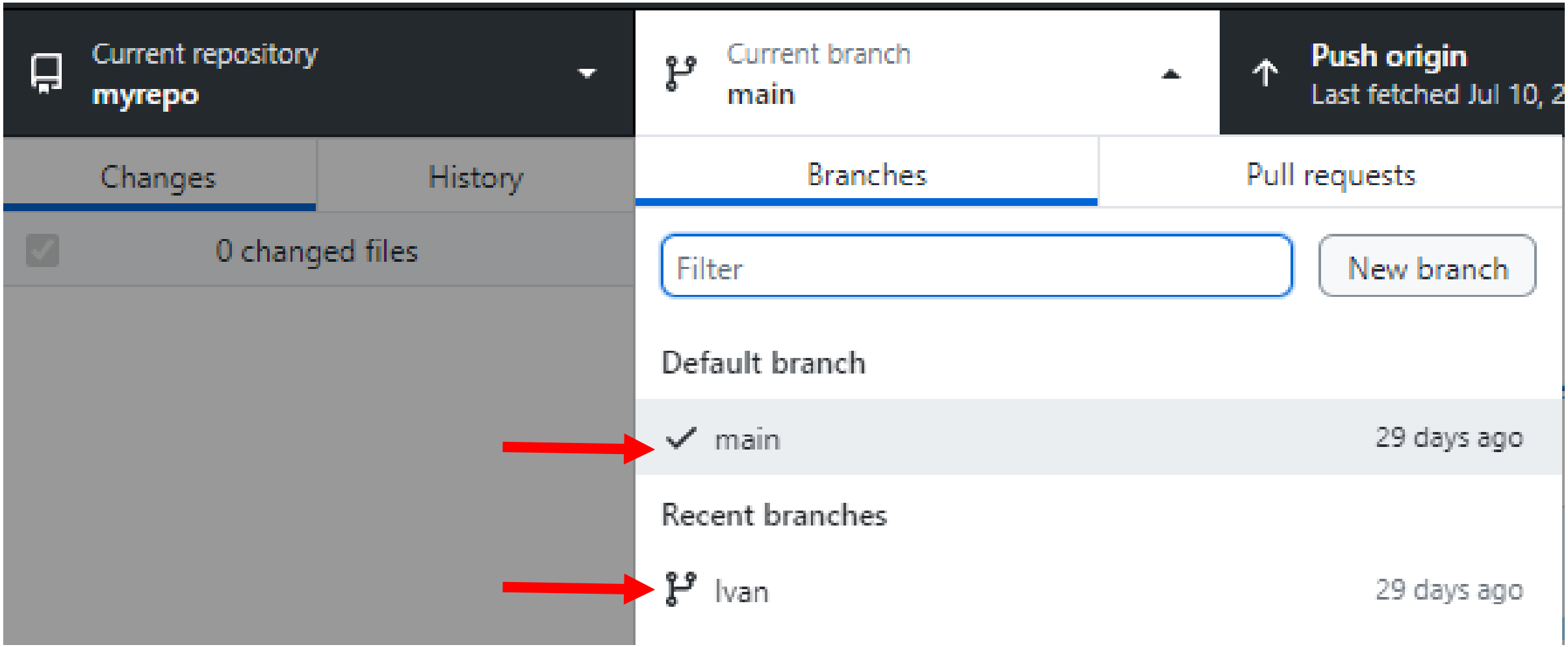
Ivan2

Create branch based on...

main
The default branch in your repository. Pick this to start on something new that's not dependent on your current branch.

Ivan
The currently checked out branch. Pick this if you need to build on work done on this branch.

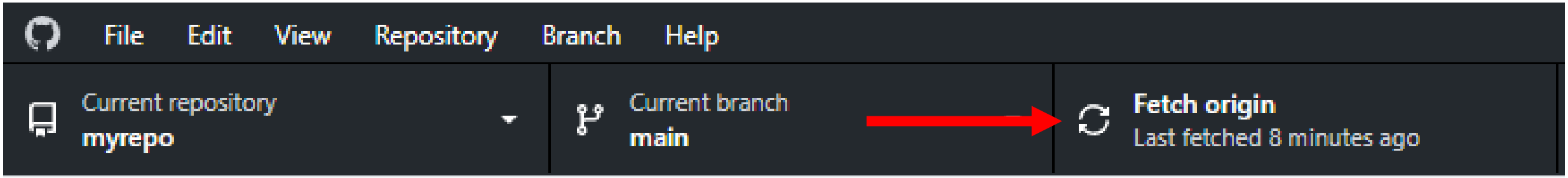
Create branch | Cancel


Current repository
myrepo
Current branch
main
Push origin
Last fetched Jul 10, 2
Changes
History
Branches
Pull requests
0 changed files
Filter
New branch
Default branch
main
29 days ago
Recent branches
Ivan
29 days ago

Обратите внимание, что когда вы меняете ветку, то файлы в вашей папке с проектом также автоматически обновляются!

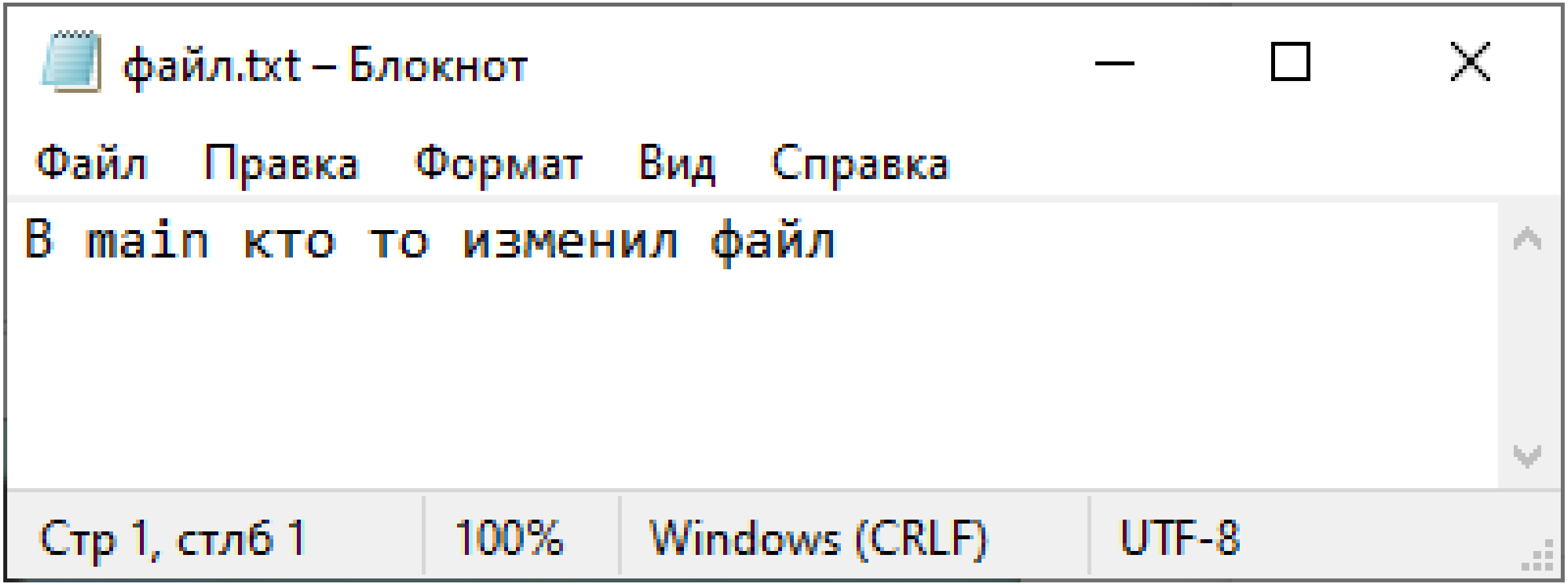
Чтобы ветка появилась в удаленном репозитории её также нужно опубликовать.

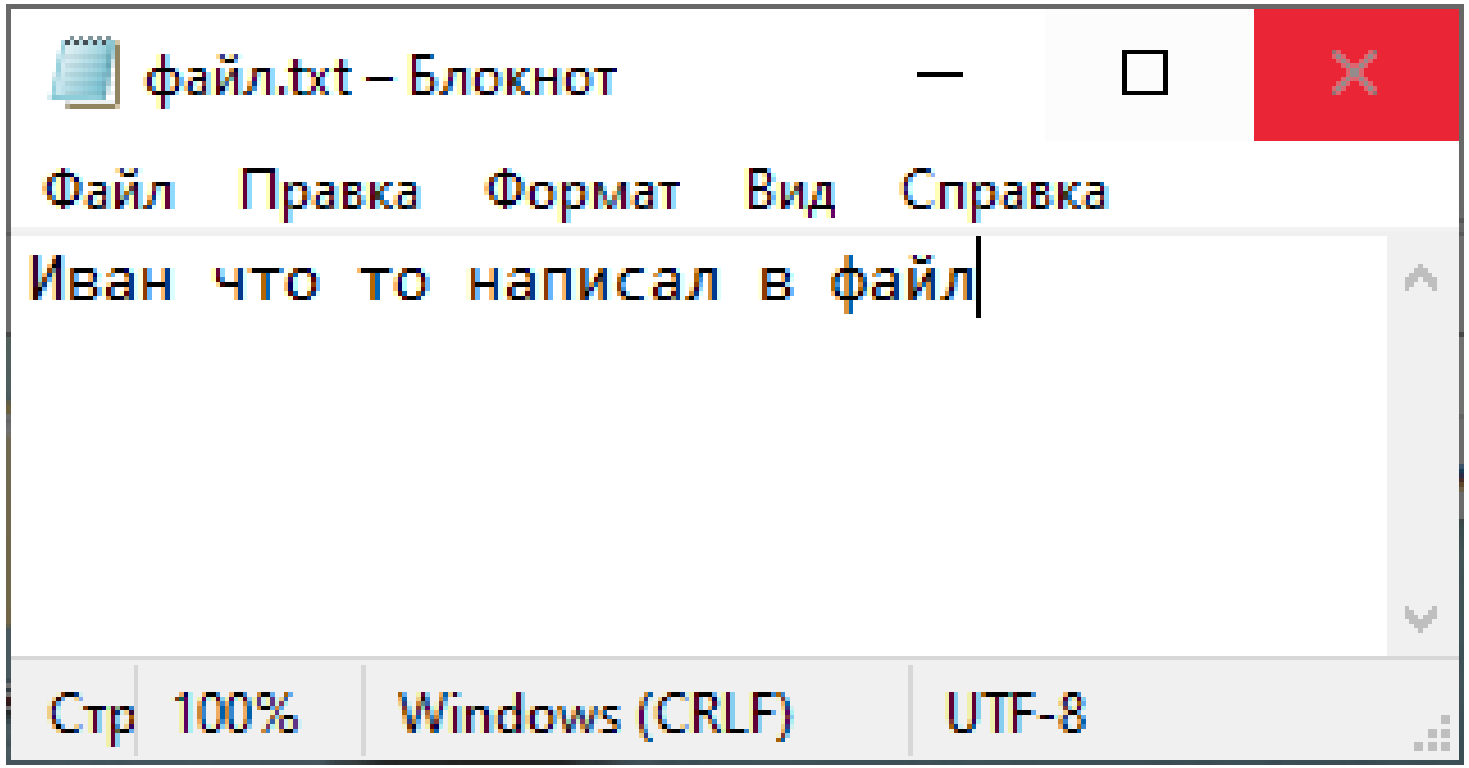
Чтобы забрать изменения с удаленного репозитория необходимо нажать Fetch (например, когда кто-то изменил файлы и вы хотите обновить свой проект в соответствии с изменениями удаленного репозитория)



Данную команду желательно выполнять перед началом работы с репозиторием.

Рассмотрим последний пример. Предположим у нас есть 2 ветки: main и Ivan, в обоих репозиториях есть файл с данным. Пусть в каждой ветке произошли изменения содержимого данного файла.

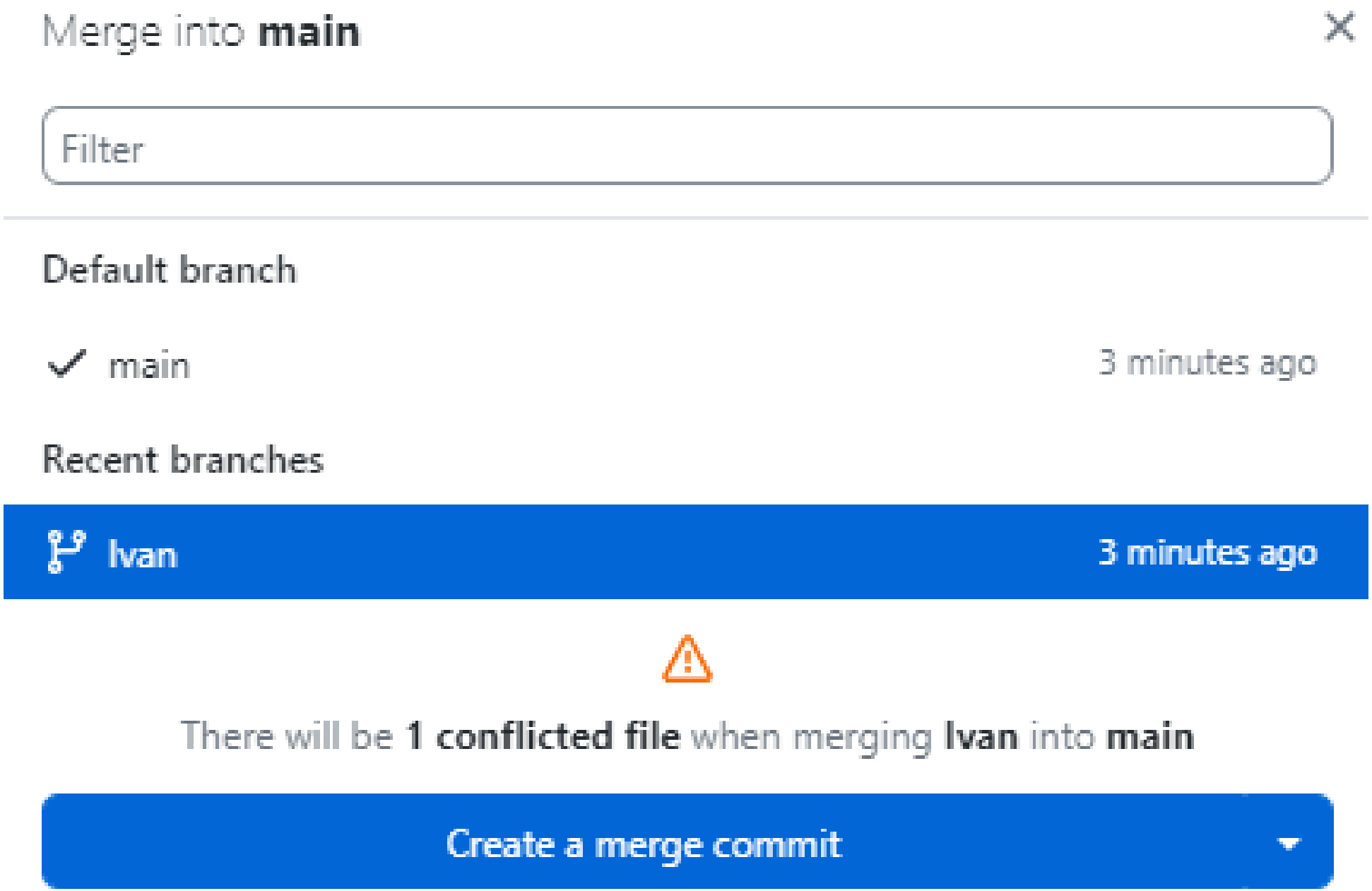




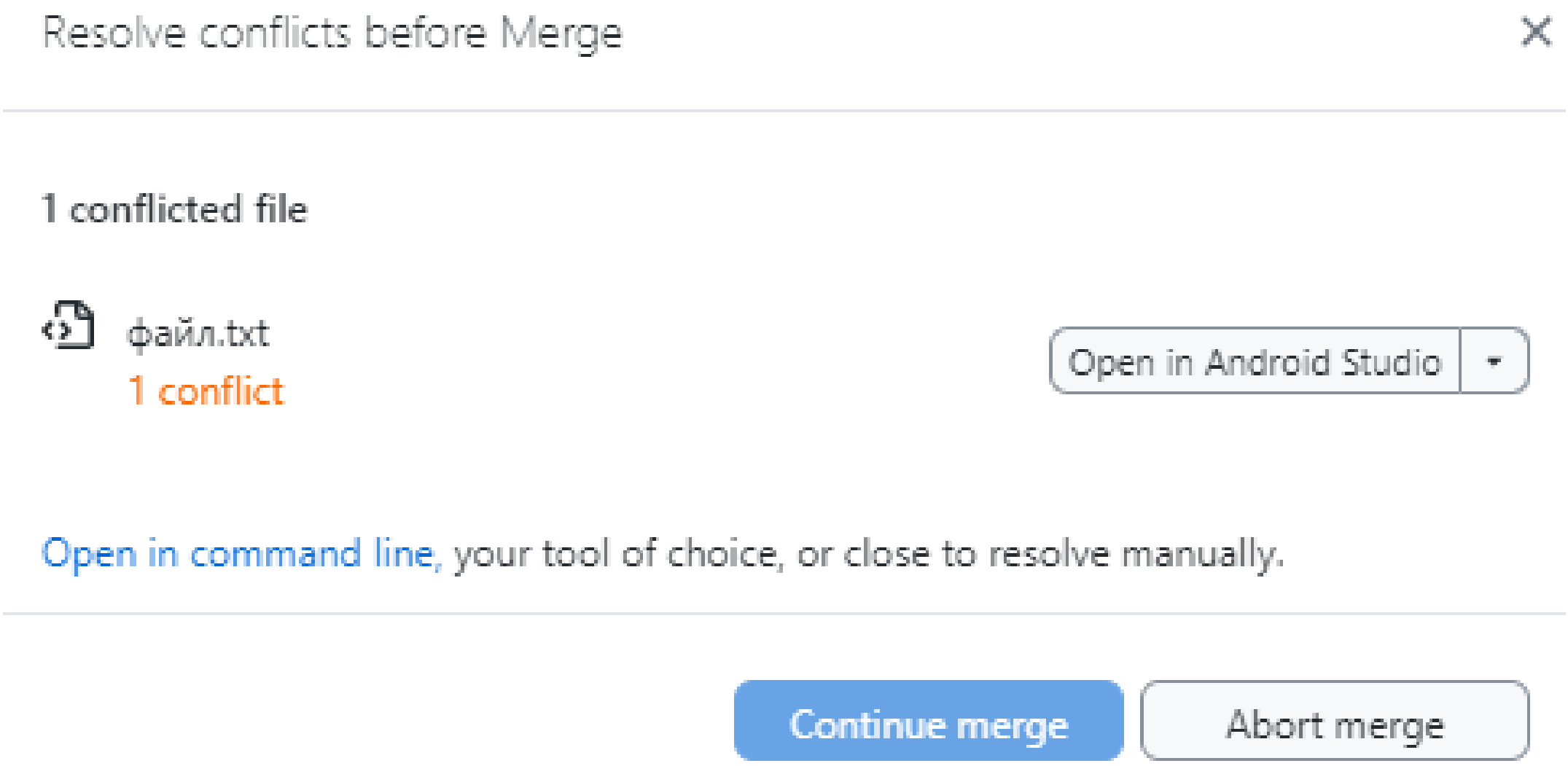
У нас есть 2 версии файла и чтобы объединить ветки, мы должны решить как будет выглядеть итоговый файл (получили конфликт слияния или merge конфликт).

Делаем слияние. Переходим на ветку main, нажимаем **Branch** -> **Merge into current branch**

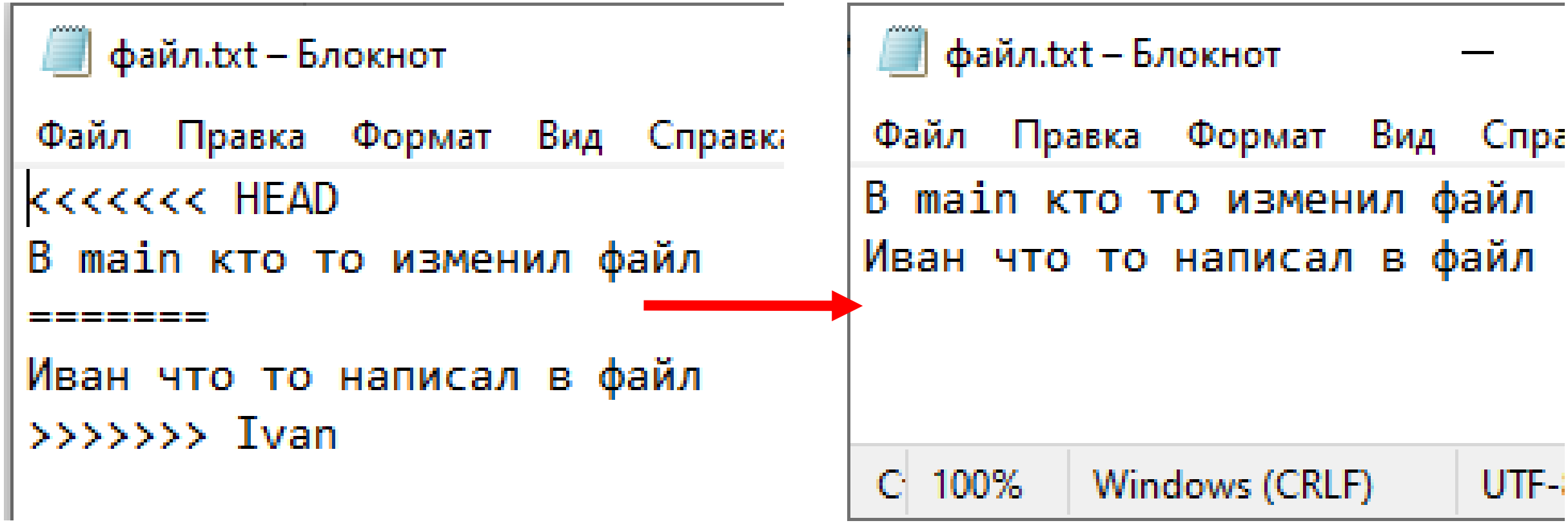
Выбираем Ivan и нажимаем **Create merge commit**



Необходимо отредактировать наш файл чтобы решить конфликт, открываем файл любой программой



Необходимо отредактировать наш файл чтобы решить конфликт, открываем файл любой программой. Конфликт считается решенным, если нет специальных символов <<<<HEAD, ===, >>>.



После чего нажимаем continue merge. Конфликт слияния решен, теперь ветка main содержит изменения Ивана.

Итог

Рассмотрим возможные при работе **состояния репозитория**:

1. Изменений нет.
2. Проводим изменения в репозитории (добавление/изменение/удаление).
3. Добавляем изменения в список для дальнейшей фиксации (сохранения) командой **git add**.
4. Фиксируем изменения с помощью команды **git commit** (данные сохраняются в локальном репозитории на ВАШЕМ компьютере).
5. Отправка изменений на удаленный сервер командой **git push** (данные сохраняются в удаленном репозитории на ДРУГОМ компьютере).

Важно помнить, что git хранит только изменения, он не хранит разные версии проектов по отдельности.

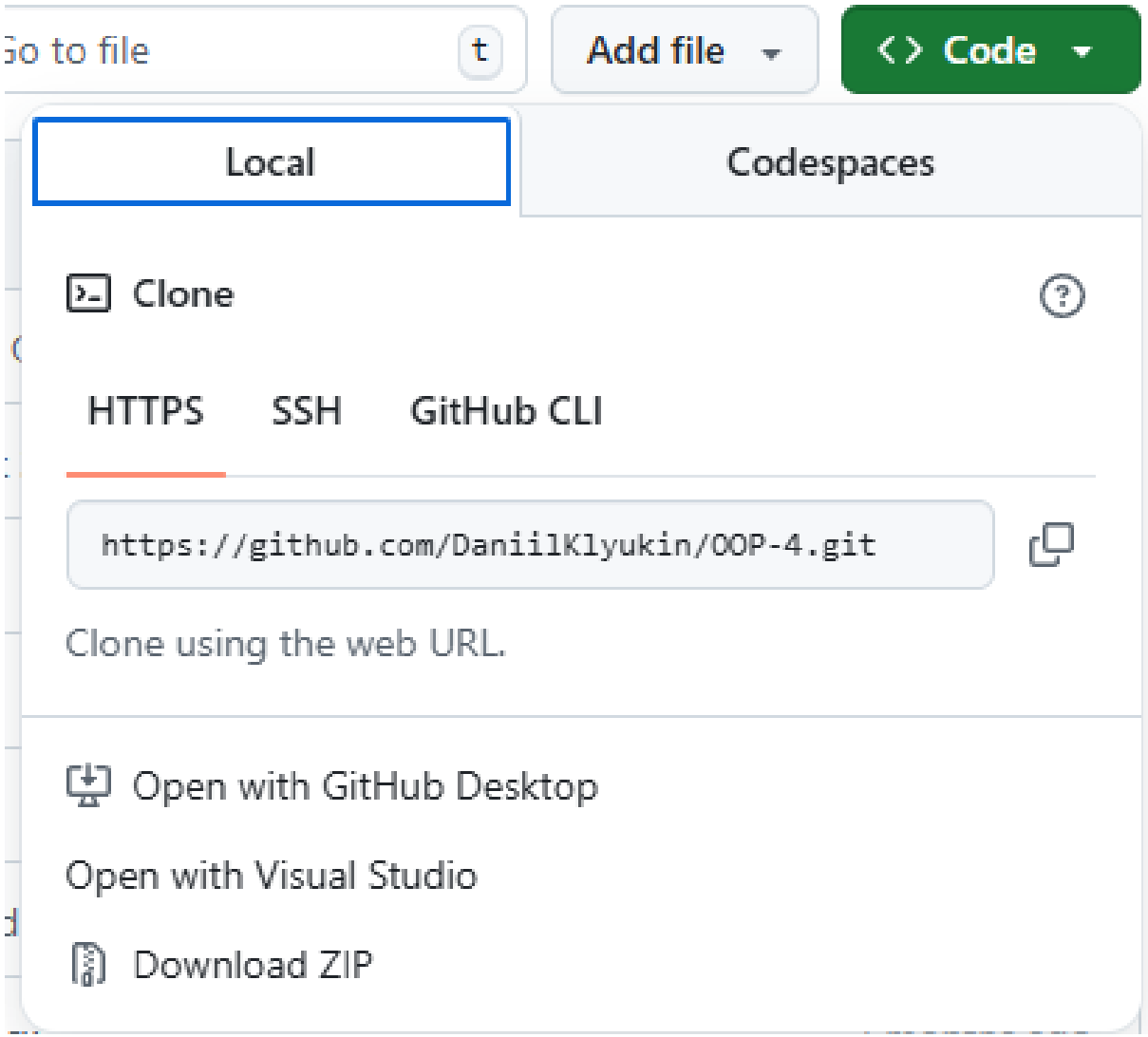
Как скачать репозиторий

Чтобы скачать репозиторий, необходимо:

1) Нажать Code
2) Download ZIP

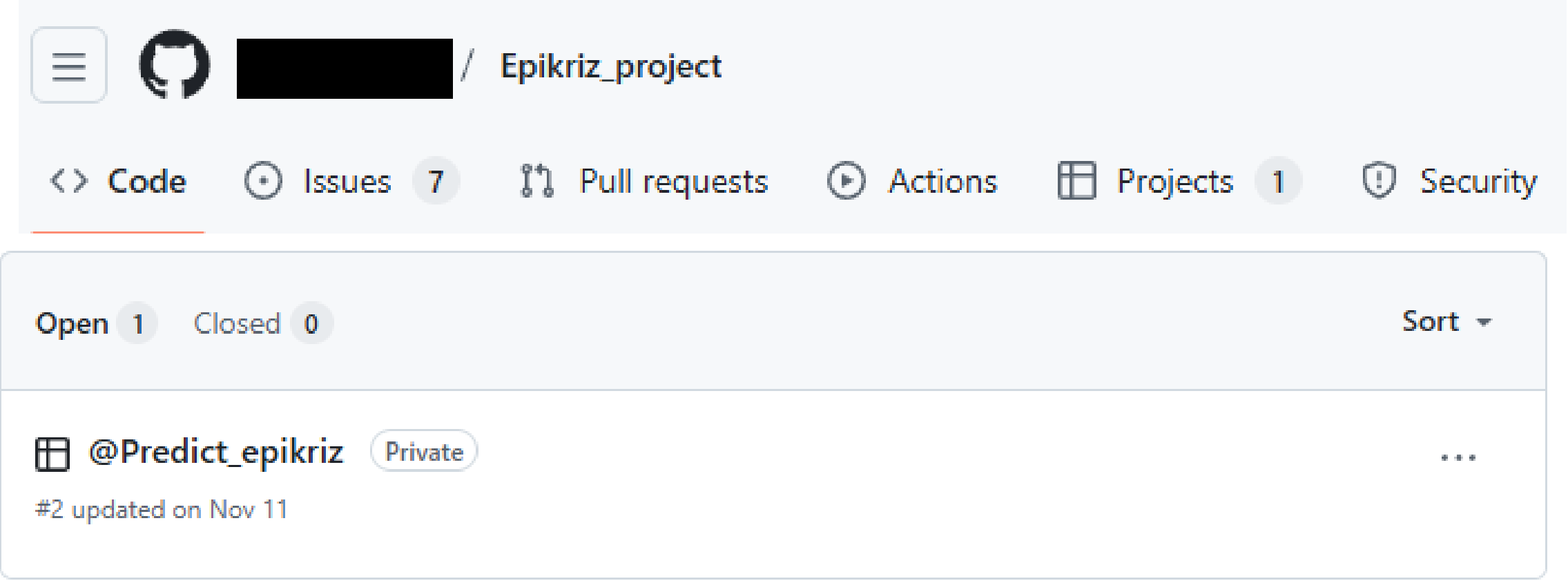
Либо клонировать:

1) Нажать Code
2) Open with GitHub Desktop
3) Clone



Ведение проекта

GitHub Projects — это гибкая доска задач (канбан), интегрированная с вашим репозиторием. Она помогает организовать работу, отслеживать прогресс и управлять сроками. Представьте себе цифровую доску со стикерами-задачами, которые можно перемещать между колонками.



@Predict_epikriz

Backlog | Team capacity | Current iteration | Roadmap | My items | + New view

Filter by keyword or by field

Todo 1 / 5 Estimate: 0

This item hasn't been started

Epikriz_project #16

Аугментация (Увеличение датасета)

In Progress 1 / 5 Estimate: 0

This is actively being worked on

Epikriz_project #23

Реализовать Удобную систему обращения к лучшей модели

Done 7 Estimate: 0

This has been completed

Epikriz_project #1

Распарсить Эпикризы

Iteration

Epikriz_project #2

Перевод данных в нужный формат

Создание проекта

Перейдите на вкладку **Projects** в вашем репозитории

Нажмите **New Project**

Выберите шаблон:

- **Basic kanban** — для простого workflow (To do / In progress / Done)
- **Automated kanban** — автоматическое перемещение задач по статусам
- **Bug triage** — для отслеживания ошибок
- **Team planning** — для планирования спринтов

Создание задачи:

Нажмите **+ Add item** в нужной колонке

Введите название задачи

Нажмите Enter

Кликните на карточку для деталей

Автоматизация (не обязательно):

Создайте Issue в репозитории с описанием задачи

В проекте нажмите + Add item и введите # + номер Issue

Карточка автоматически свяжется с Issue

При создании Pull Request с ключевыми словами:

fixes #1 — автоматически переместит карточку в "Done"

connects to #1 — свяжет PR с задачей